

构建高效的智能体

2026.07.22



目录

CONTENTS

01 访谈诊断

02 解决方案

03 预期目标

04 商务报价

什么是智能体

“智能体（Agent）”存在多种定义方式。一种是将智能体理解为具备完全自主能力的系统，能够长时间独立运行，调用各类工具完成复杂任务；另一种则是用术语来约束更强的实现方案，这类方案遵循预设 workflow 执行。两者统称为智能体系统（agentic systems）。

从架构上明确划分 workflow 与智能体：

- **工作流**依靠预设代码链路调度大模型与工具
- **智能体**由大模型动态决定执行步骤与工具调用，自主把控任务执行方案

什么时候用或不用智能体

基于大模型开发应用，建议优先选用最简方案，仅按需增加复杂度，甚至不必构建智能体系统。智能体通常以牺牲响应速度、增加运行成本来提升任务效果，应当判断该取舍或平衡好选择。

当方案确实变得越复杂时：

- 对于边界明确的任务，工作流能提供稳定性、可预期性。
- 当规模化场景需要灵活性与模型自主决策时，则优先选用智能体。
- 不过多数应用只需通过检索与上下文示例优化单次大模型调用即可。

方案选型优先级（由简到繁）：

单次 LLM 调用（RAG + 上下文示例） → 固定工作流 Workflow → 自主智能体 Agent

框架的适用场景与使用方式

1. 多款框架可简化智能体系统开发，包括：

- Claude Agent SDK
- AWS Strands Agents SDK
- Rivet：拖拽式可视化 LLM workflow 搭建工具
- Vellum：可视化复杂 workflow 搭建与测试工具

2. 框架的好处：

- 能简化各类底层操作，如简化了调用大模型、定义与解析工具、串联模型调用等底层常规任务，便于快速开发。
- 但额外带来的抽象层会隐藏原始提示词与返回内容，增加调试成本。
- 同时框架容易诱导开发者过度设计其实在很多场景下轻量化配置即可解决的问题。

框架使用方式

3. 建议开发者：

- 先直接调用大语言模型 API 进行开发：许多范式仅需少量代码即可实现。
- 如果确实要使用框架，务必理解其底层代码。
- 不了解框架内核而产生误判，是常见的出错原因。

基础组件、工作流与智能体

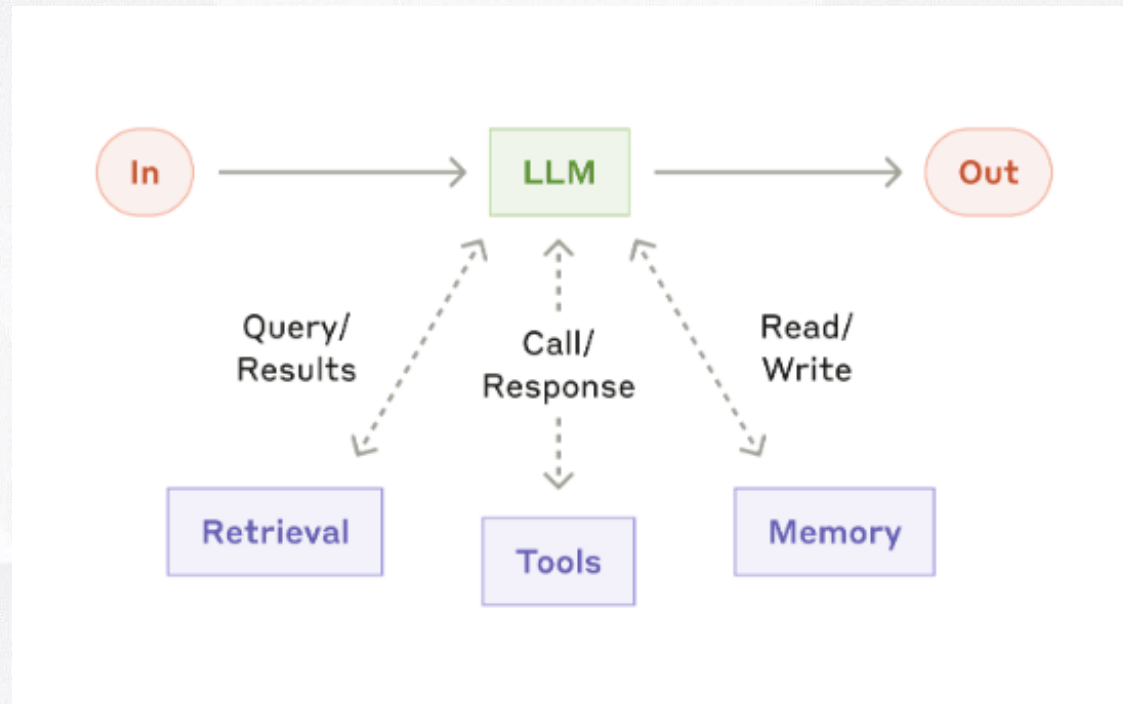
基础组件(Building blocks): 最底层可复用单元

工作流(Workflows): 由基础组件按固定链路组合而成

智能体(Agent)s: 依托基础组件, 具备动态自主调度能力

基础组件: 增强型大语言模型

- 智能系统的基础单元为增强型大语言模型，支持检索、工具调用、记忆等能力。
- 现有模型可主动运用上述能力：自主构造检索语句、选择工具、决定信息留存范围。



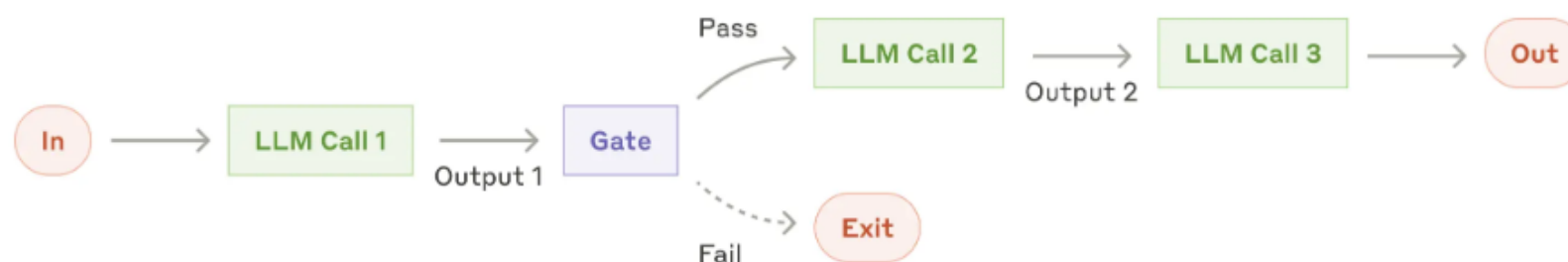
基础组件: 增强型大语言模型

建议开发重点聚焦两大核心方向：

- 一是针对自身业务场景定制上述基础组件的扩展能力。
- 二是为大语言模型提供简洁完善、文档齐备的交互接口。
- 实现这类增强能力的方案有很多种，其中一种可以借助模型上下文协议（Model Context Protocol）
开发者只需实现简易客户端，就能接入持续扩容的第三方工具生态。

workflows: 提示词链式调用

- 提示词链式调用把任务拆分为串行步骤，每轮 LLM 调用承接上一轮输出。可在各中间节点加入代码校验（图中 Gate），保障流程不偏离目标。



提示词链式调用 workflow

串行执行模式；支持在链路中插入代码判断，不满足条件时可终止或分支处理。

workflow: 提示词链式调用适用场景

- 该 workflow 非常适合能够清晰、规整地拆解为若干固定子任务的场景。
- 其核心思路是以延迟为代价换取更高准确率，将每一轮大模型调用的任务复杂度降低。

适合采用提示词链式调用的场景示例：

- 生成营销文案，再将文案翻译成其他语言。
- 起草文档大纲，校验大纲是否符合指定标准，随后依据大纲撰写完整文档。

流程为典型串行依赖，后一步必须依赖前一步结果，非常契合提示词链式调用的形态。

workflow: 路由分发

- 路由 workflow 会对输入内容进行分类，并将其分发至对应的专项后续任务。
- workflow 能够实现关注点分离，同时构建更具针对性的提示词。
- 倘若不采用这种模式，针对某一类输入进行优化，往往会损害模型处理其他类型输入的效果。

路由分发 workflow



workflow: 路由分发适用场景

- 路由 workflow 适用于含有清晰类别划分的复杂任务，各类请求适合分开处理，并且可以通过大语言模型或传统分类模型、算法实现精准分类。

路由模式适用示例：

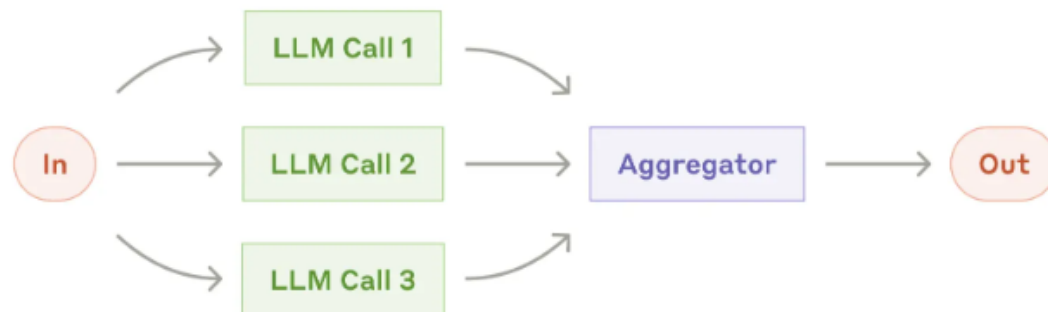
- 将不同类型的客服咨询（通用问询、退款申请、技术支持）分发至对应的下游流程、提示词与工具；
- 把简单、常见问题路由至轻量化高性价比模型（如 Claude Haiku 4.5），复杂、非常规问题交由能力更强的模型（如 Claude Sonnet 4.5），兼顾综合效果与成本。

workflows: 并行处理

多个大语言模型调用可同步处理任务，再通过代码聚合各自输出。这种并行处理 workflow 主要分为两种典型形式：

- 任务拆分并行 (Sectioning)：将任务拆解为相互独立的子任务，并发执行。
- 多轮投票 (Voting)：多次运行同一任务，获取多样化输出结果。

并行处理 workflow



workflows: 并行处理适用场景

以下场景可让并行 workflow 发挥显著作用：

- 拆分后的子任务可并发执行以提升处理速度，缩短耗时。
- 需要多维度分析、多次生成结果来提高结论可信度。
- 需要多重考量的复杂任务，将每项考量点交由独立的大模型调用单独处理，让模型专注分析单一维度，整体输出效果通常会更优。

并行处理适用示例

任务分片并行 (Sectioning)

- 配置内容安全防护机制：一路模型处理用户提问，另一路同步筛查违规内容与不当请求。该方案效果优于单轮大模型调用同时完成内容校验与核心回答生成。
- 自动化模型评测：多组大模型调用分别从不同维度，评测同一提示词下模型的各项表现。

投票并行 (Voting)

- 代码漏洞审计：使用多套独立提示词同步审查代码，一旦任意链路检测出风险便标记问题。
- 内容合规判定：多套提示词分别从不同维度评估内容是否违规，搭配差异化投票阈值，平衡误判（误伤正常内容）与漏判（放过违规内容）问题。

workflow: 调度器 - 工作单元模式

由中心大模型动态拆解整体任务，将子任务分派给各执行大模型，最后统一汇总全部结果。

调度器 - 工作单元 workflow



workflow: 调度器 - 工作单元模式适用场景

- 该 workflow 非常适合无法提前预判所需子任务的复杂场景（例如代码开发场景：需要修改的文件数量、各文件的修改内容，都随具体任务变化）。
- 虽然它在执行形态上和并行处理模式相近，但二者核心差异在于灵活性：本模式的子任务并非预先定义，而是由调度中枢根据具体输入实时生成。

适用示例：

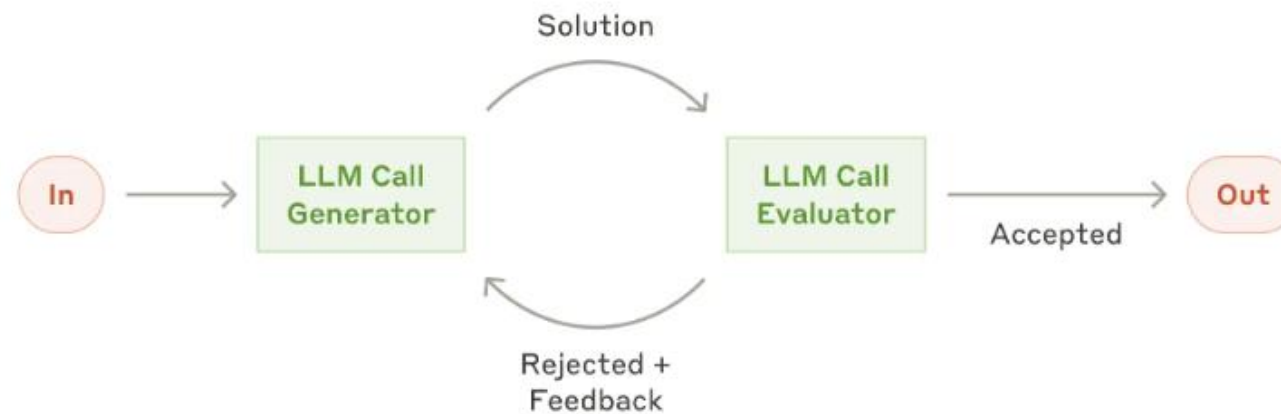
- 代码开发工具，每次需对多个文件执行复杂修改操作。
- 信息检索任务，需要从多渠道收集、分析资料，筛选有效相关信息。

workflows: 评估 - 迭代优化模式

评估 - 优化器 workflow 机制:

一轮大模型调用生成输出内容, 另一轮大模型负责给出评估意见与优化反馈, 二者循环迭代执行。

评估 - 优化迭代 workflow



workflows: 评估 - 迭代优化模式适用场景

- 当存在清晰的评估标准，且迭代优化能够带来可量化的效果提升时，该 workflow 效果尤为突出。
- 适配该模式有两大核心特征：
 - 第一，人为给出明确反馈后，大模型的输出质量能够得到显著改善。
 - 第二，大模型自身有能力输出这类评价反馈。
- 这一机制与人撰写精美文稿时反复修改打磨的迭代写作过程原理相通。

智能体及其工作机制

- 智能体是建立在大语言模型在核心能力不断成熟的基础上如：**理解复杂输入、自主推理和规划、稳定调用工具、处理执行异常**
- 智能体的启动方式分为两种：接收用户指令，或与用户开展交互式对话
- 明确任务目标后，智能体会自主规划并执行流程，必要时主动向人类寻求补充信息或决策判断。
- 执行过程中，智能体每一步都必须从外部环境获取真实基准信息（例如工具调用返回结果、代码运行输出），以此判断当前任务进度。
- 智能体可在关键节点、或是遭遇执行障碍时暂停运行，等待人类反馈。任务通常在完成后自动结束。
- 工程实践中一般会设置终止条件（例如最大迭代次数），以此管控运行范围、避免无限执行。

智能体适用场景

- 智能体适用于开放式问题，很难甚至无法预判所需执行步骤，也无法写死固定处理流程。
- 智能体可能会进行多轮循环操作，因此使用者需要对其决策能力具备一定信任度。
- 依托自主运行特性，智能体非常适合在可信环境中批量处理规模化任务。

注意事项：

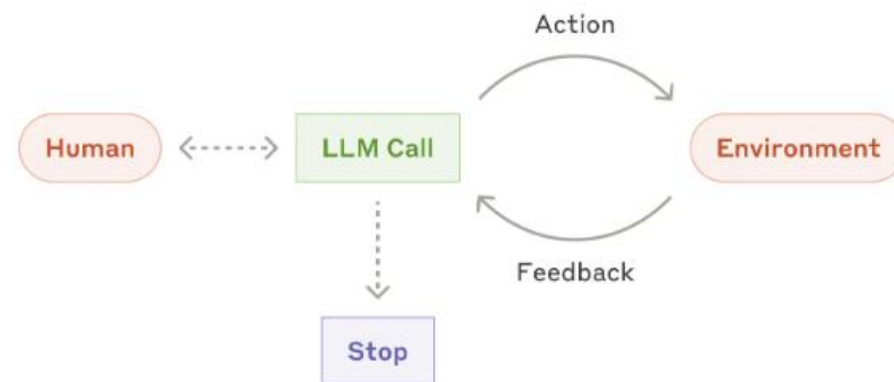
- 智能体具备自主运行的特性，随之带来更高的使用成本
- 同时存在误差持续累积放大的风险
- 建议先在隔离沙箱环境中完成充分测试，并配套部署适配的内容安全防护机制。

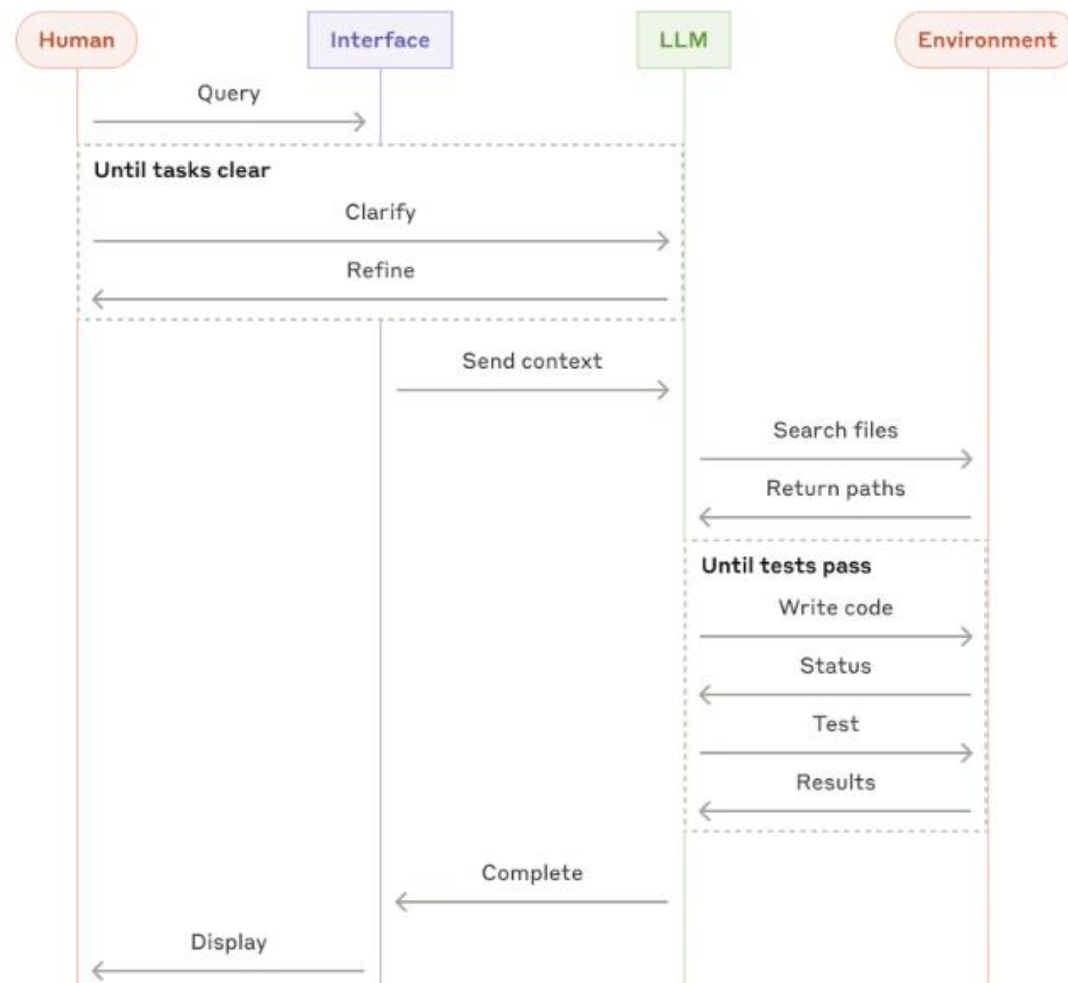
智能体适用示例

以下案例均来自我们自研落地的项目：

- 代码智能体：批量修改多文件，完成 SWE-bench 工程测试任务
- 电脑操控智能体：Claude 直接操作计算机完成复杂任务

自主智能体





- 尽管各类开发框架能够帮助快速上手搭建原型，但在向生产环境迁移时，建议适当精简抽象层级，改用基础组件自主搭建。
- 坚守三大设计原则，打造出的智能体不仅功能强大，同时具备高可靠性、易维护性，也能获得用户信赖。

基础组件: 增强型大语言模型

基础组件(Building blocks): 最底层可复用单元

工作流(Workflows): 由基础组件按固定链路组合而成

智能体(Agent)s: 依托基础组件, 具备动态自主调度能力

01

访谈诊断

02

解决方案

选择多样性团队

- 选择多样性或异质化团队而不是同质化团队（大多数时候）



谢谢

THANK YOU